

Expense Tracker Documentation

Project Title

Expense Tracker Using Python

Objective

The objective of this project is to create a simple expense tracking program that allows users to enter multiple expense amounts and calculates the total amount spent. The program demonstrates the use of accumulators, loops, exception handling, and sentinel values in Python.

Problem Statement

Users often need a simple way to track their daily expenses. This program enables users to enter expense amounts one by one and continuously updates the total expenditure. The program stops accepting input when the user enters a specific keyword (`quit`) and displays the final total spent.

Features

- Accepts multiple expense transactions.
 - Calculates and displays the running total.
 - Handles invalid inputs using exception handling.
 - Uses a sentinel value (`quit`) as a kill switch.
 - Displays the final total before exiting the program.
 - Supports 5 or more transactions.
-

Requirements

- Python 3.x
-

Program Logic

1. Initialize the variable `total` to 0.
2. Display instructions to the user.
3. Continuously prompt the user to enter an expense amount.
4. If the user enters `"quit"`:
 - Display the final total spent.
 - Exit the program.
5. Convert the input to a floating-point number.
6. If conversion fails:
 - Display an error message.
 - Ask for input again.
7. Add the valid expense amount to `total`.

8. Display the updated total.
9. Repeat the process until the user enters "quit".

Source Code

```
total = 0

print("Enter expense amounts one by one. Type 'quit' to stop and show the total.")

print("You can enter 5 or more transactions.")

while True:
    new_expense = input("Enter expense amount or 'quit': ")
    if new_expense.lower() == "quit":
        print(f"The Final Total is {total}. Exiting...")
        break
    try:
        new_expense = float(new_expense)
    except ValueError:
        print("Invalid input. Please enter a numeric expense amount.")
        continue
    total += new_expense
    print(f"Added {new_expense:.2f}. Total Spent so far: {total:.2f}")
```

Variables Used

Variable	Data Type	Description
total	Float	Stores the cumulative total of all expenses
new_expense	String / Float	Stores user input and converted expense value

Key Concepts Implemented

1. Accumulator

The variable `total` acts as an accumulator by continuously adding new expense values.

```
total += new_expense
```

2. While Loop

The program uses a `while True` loop to repeatedly accept user input until the user decides to quit.

```
while True:
```

3. Sentinel Value (Kill Switch)

The word "quit" is used as a sentinel value to terminate the program.

```
if new_expense.lower() == "quit":
```

4. Exception Handling

The program handles invalid inputs using a try-except block.

```
try:
    new_expense = float(new_expense)
except ValueError:
    print("Invalid input. Please enter a numeric expense amount.")
```

Sample Execution

Input

```
Enter expense amount or 'quit': 100
Enter expense amount or 'quit': 50
Enter expense amount or 'quit': 20
Enter expense amount or 'quit': quit
```

Output

```
Added 100.00. Total Spent so far: 100.00
Added 50.00. Total Spent so far: 150.00
Added 20.00. Total Spent so far: 170.00
The Final Total is 170.0. Exiting...
```

Requirement Verification

Requirement	Status
Handle 5+ transactions	✓ Implemented
Catch <code>ValueError</code> on input	✓ Implemented
Total initialized outside the loop	✓ Implemented
Kill switch prints final total	✓ Implemented

Conclusion

The Expense Tracker program successfully records multiple expense transactions, calculates the total amount spent, handles invalid inputs gracefully, and terminates using a sentinel value. The project demonstrates fundamental Python concepts such as loops, accumulators, exception handling, and user input processing.